

NOTES

Inheritance and Polymorphism



Polymorphism and Inheritance

Polymorphism in Object-Oriented Programming (OOP)

What is Polymorphism?

Polymorphism, derived from the Greek words "poly" (many) and "morph" (form), is a fundamental concept in Object-Oriented Programming (OOP) that allows objects of different classes to be treated as if they are instances of a common superclass. This capability enables a common interface for multiple types, promoting code flexibility, modularity, and scalability. Polymorphism simplifies complex systems by enabling functions to interact with objects of different types in a standardised way. In Python and other programming languages, polymorphism can be categorised into two main types: **Compile-Time** Polymorphism and **Run-Time** Polymorphism.

Compile-Time Polymorphism (Static)

Compile-time polymorphism, also known as static polymorphism or early binding, occurs when the method to be executed is determined at compile time. This type of polymorphism is primarily achieved through method overloading and operator overloading.

Example:

```

Python
class Calculator:
    # Method for adding two integers
    def add(self, a: int, b: int) -> int:
        return a + b

    # Method for adding three integers
    def add(self, a: int, b: int, c: int) -> int:
        return a + b + c

# Create an instance of Calculator
calc = Calculator()

# Call the add method
print(calc.add(10, 20))      # Output: 30 (This will cause an error
                             # due to method overriding)
print(calc.add(10, 20, 30))  # Output: 60 (This will also cause an
                             # error)
  
```

Note: In Python, method overloading does not work in the same way as in languages like Java or C++. The last defined method will override any previous definitions. Therefore, this example serves to illustrate the concept but will not function correctly in Python.

Characteristics:

- The compiler resolves the method call during compilation.
- Less flexible compared to run-time polymorphism since the method to be executed is known before execution.
- Commonly used when methods with the same name differ by parameter type or number.

Example: Method Overloading

Run-Time Polymorphism (Dynamic)

Run-time polymorphism, also known as dynamic polymorphism or late binding, occurs when the method to be executed is determined at run time. This type of polymorphism is primarily achieved through method overriding.

Example:

```
Python
class Animal:
    def speak(self):
        return "Animal speaks"

class Dog(Animal):
    def speak(self):
        return "Woof!"

class Cat(Animal):
    def speak(self):
        return "Meow!"

# Create instances of Dog and Cat
animals = [Dog(), Cat()]

# Call the speak method for each animal
for animal in animals:
    print(animal.speak()) # Output: Woof! Meow!
```

Characteristics:

- The decision about which method to invoke is made at runtime based on the object type.
- More flexible than compile-time polymorphism as it allows for dynamic method resolution.
- Commonly used in inheritance hierarchies where a child class overrides a method from its parent class.

Example: Method Overriding

Why Use Polymorphism?

Code Flexibility: Allows you to use the same interface for different underlying forms (data types).

Example: In this example, both **Dog** and **Cat** classes implement the **speak()** method, allowing us to treat different objects (Dog and Cat) in a uniform way.

```
Python
class Animal:
    def speak(self):
        pass

class Dog(Animal):
    def speak(self):
        return "Woof!"

class Cat(Animal):
    def speak(self):
        return "Meow!"

# Function that takes any Animal and calls its speak() method
def animal_sound(animal):
    print(animal.speak())

# Using the same function interface for different underlying forms
animal_sound(Dog()) # Output: Woof!
animal_sound(Cat()) # Output: Meow!
```

The **animal_sound()** function is flexible and can take any object that has a **speak()** method, regardless of the underlying class type, as long as they inherit from **Animal**.

Extendability: Enables the addition of new classes and functionality without altering existing code.

Example: By adding new classes like `Bird`, we can extend our code to include new types of animals without changing the `animal_sound` function.

```
Python
class Bird(Animal):
    def speak(self):
        return "Chirp!"

# Reusing animal_sound without modifying it
animal_sound(Bird()) # Output: Chirp!
```

Since each class can have its own version of `speak()`, we can add new classes (like `Bird`) that don't require us to update the `animal_sound` function, which makes our code more extendable.

Simplifies Code: Reduces conditional branching in functions by allowing each object to define its unique behaviour.

Example: Instead of using conditional statements to check the type of each object, each class defines its own behaviour, simplifying the main code logic.

```
Python
class Fish(Animal):
    def speak(self):
        return "Blub!"

animals = [Dog(), Cat(), Bird(), Fish()]

for animal in animals:
    print(animal.speak())
```

Without polymorphism, we might need to check each object type explicitly, like `if isinstance(animal, Dog): print("Woof!")`. With polymorphism, the `speak()` method call automatically invokes the correct behaviour for each type, which makes the code cleaner and more maintainable.

Core Concepts in Polymorphism

Method Overriding

Method overriding occurs when a child class provides its own implementation of a method that is inherited from the parent class. This allows the child class to tailor the method to suit its specific needs while still maintaining a unified interface.

Example: In an animal simulation, different animal types might make distinct sounds. Here, an "Animal" parent class has a `speak` method, and child classes like `Dog` and `Cat` override `speak` to reflect their specific sounds.

```
Python
class Animal:
    def speak(self):
        return "Animal sound"

class Dog(Animal):
    def speak(self):
        return "Woof!"

class Cat(Animal):
    def speak(self):
        return "Meow!"

# Function that calls speak method
def animal_sound(animal):
```

```
print(animal.speak())  
  
animal_sound(Dog()) # Output: Woof!  
animal_sound(Cat()) # Output: Meow!
```

Method overriding enables customised behaviour for child classes without changing the function interface. Thus, the `animal_sound` function can operate on any `Animal` instance, regardless of its specific subclass.

The **parent class** defines a method with a certain name and behaviour, and the **child class** redefines (overrides) this method to provide its unique implementation.

Why is Method Overriding Useful?

- **Customised Behaviour:** Allows child classes to define behaviour that's specific to them without altering the method signature.
- **Unified Interface:** Methods can be called consistently across all child classes, enhancing code readability and simplicity.

Method Overloading (via Default Arguments)

What is Method Overloading?

Method overloading traditionally means defining multiple methods with the same name but different parameters. However, Python doesn't support traditional method overloading. Instead of overloading, Python uses **default arguments** to allow a single method to accept a variable number of arguments.

By setting default values for parameters, a single method can perform different tasks based on the number of arguments passed.

Example: Imagine a **Calculator** class where the **add** method can add two or three numbers depending on the input.

```
Python
class Calculator:
    def add(self, a, b=0, c=0):
        return a + b + c

# Creating an instance of Calculator
calc = Calculator()

# Using add method with different numbers of arguments
print(calc.add(5))           # Only 1 argument, Output: 5
print(calc.add(5, 10))       # 2 arguments, Output: 15
print(calc.add(5, 10, 15))   # 3 arguments, Output: 30
```

Python doesn't support traditional method overloading (multiple methods with the same name but different parameters). Instead, we use default arguments to allow a single method to handle different numbers of arguments. Here, the **add** method in the **Calculator** class is defined with three parameters (**a**, **b**, and **c**), where **b** and **c** have default values of 0.

- **When calling** `calc.add(5)`, only one argument is provided, so **b** and **c** use their default value of 0. The result is $5 + 0 + 0 = 5$.
- **When calling** `calc.add(5, 10)`, two arguments are provided, so **a** is 5 and **b** is 10, with **c** still defaulting to 0. The result is $5 + 10 + 0 = 15$.
- **When calling** `calc.add(5, 10, 15)`, all three arguments are provided, so the result is $5 + 10 + 15 = 30$.

Why is Method Overloading via Default Arguments Useful?

- **Flexible Function Calls:** A single method can perform different tasks depending on the number of arguments provided, making the code more versatile.
- **Simplicity and Maintenance:** No need to define multiple methods; using default arguments makes the code easier to read and maintain.



Inheritance in Object-Oriented Programming (OOP)

What is Inheritance?

Inheritance is a fundamental concept in Object-Oriented Programming (OOP) that allows a class to inherit properties and methods from another class. The class that inherits is called the **child** (or **derived**) class, and the class it inherits from is the **parent** (or **base**) class.

Why Use Inheritance?

Code Reusability

- Saves time and effort by reusing code from the parent class in multiple child classes.
- **Example:** If you create a class for a **Vehicle** with methods like **start** and **stop**, then child classes like **Car** or **Bike** can inherit these methods without needing to rewrite them.

Organised Hierarchy

- Groups similar classes together, creating a logical structure in the code.
- **Example:** A **Shape** class might have child classes **Circle**, **Square**, and **Rectangle**, which helps organise properties and functions related to shapes.

Easier Maintenance

- Simplifies updates and fixes because shared functionalities are centralised in the parent class.
- **Example:** If you update a method in the **Vehicle** class, the changes automatically apply to all vehicles (e.g., **Car**, **Bike**, **Truck**), so you don't need to update each type individually.

Real-World Example:

School System:

- **Parent Class: Person**
 - Properties: **name**, **age**, **address**
 - Methods: **introduce_self()**
- **Child Classes:**
 - **Student:** Inherits **name**, **age**, and **address** from **Person** and adds properties like **grade**, **student_id**.
 - **Teacher:** Inherits **name**, **age**, and **address** and adds properties like **subject**, **teacher_id**.
- **How It Works:** Both students and teachers share basic information (like name and age) from **Person** but also have unique attributes and methods specific to their roles.

```

Python
# Parent Class
class Person:
    def __init__(self, name, age, address):
        self.name = name
        self.age = age
        self.address = address

    def introduce_self(self):
  
```

```

        print(f"Hi, my name is {self.name}, I am {self.age} years
old.")

# Child Class - Student
class Student(Person):
    def __init__(self, name, age, address, grade, student_id):
        super().__init__(name, age, address)
        self.grade = grade
        self.student_id = student_id

# Child Class - Teacher
class Teacher(Person):
    def __init__(self, name, age, address, subject, teacher_id):
        super().__init__(name, age, address)
        self.subject = subject
        self.teacher_id = teacher_id

# Usage
student = Student("Alice", 16, "123 Main St", "10th", "S001")
teacher = Teacher("Mr. John", 40, "456 Park Ave", "Math", "T123")

student.introduce_self() # Inherited method
print(f"Student ID: {student.student_id}, Grade: {student.grade}")
teacher.introduce_self() # Inherited method
print(f"Teacher ID: {teacher.teacher_id}, Subject:
{teacher.subject}")

```

Transportation System

Parent Class: Vehicle

- Properties: `make`, `model`, `year`
- Methods: `start()`, `stop()`

Child Classes:

- **Car**: Inherits from **Vehicle** and adds properties like `number_of_doors`, `trunk_size`.

- **Bicycle:** Inherits from **Vehicle** and adds properties like `gear_count`, `type_of_brakes`.

How It Works: All vehicles share common attributes and methods from the **Vehicle** class. Each specific vehicle type (car, bicycle) has additional unique attributes.

Python

Parent Class

class Vehicle:

```
def __init__(self, make, model, year):
    self.make = make
    self.model = model
    self.year = year
```

```
def start(self):
```

```
    print(f"{self.make} {self.model} starting...")
```

```
def stop(self):
```

```
    print(f"{self.make} {self.model} stopping...")
```

Child Class - Car

class Car(Vehicle):

```
def __init__(self, make, model, year, number_of_doors,
trunk_size):
```

```
    super().__init__(make, model, year)
    self.number_of_doors = number_of_doors
    self.trunk_size = trunk_size
```

Child Class - Bicycle

class Bicycle(Vehicle):

```
def __init__(self, make, model, year, gear_count,
type_of_brakes):
```

```
    super().__init__(make, model, year)
    self.gear_count = gear_count
    self.type_of_brakes = type_of_brakes
```

Usage

```
car = Car("Toyota", "Camry", 2020, 4, "500L")
```

```
bicycle = Bicycle("Giant", "Escape 3", 2022, 21, "Disc")
```

```
car.start() # Inherited method
print(f"Doors: {car.number_of_doors}, Trunk Size: {car.trunk_size}")
bicycle.start() # Inherited method
print(f"Gear Count: {bicycle.gear_count}, Brakes:
{bicycle.type_of_brakes}")
```

Types of Inheritance in Object-Oriented Programming (OOP)

Single Inheritance

In single inheritance, a class inherits from only one parent class. This type keeps the inheritance structure simple, as each child class has a direct relationship with a single parent class.

Benefits:

- Reduces complexity, making it easy to understand and follow.
- Useful when a clear parent-child relationship exists with only one base class.

Example:

- **Parent Class:** `Animal`
- **Child Class:** `Dog` inherits from `Animal`

```
Python
# Parent Class
class Animal:
    def speak(self):
        print("Animal sound")

# Child Class
class Dog(Animal):
```

```
def bark(self):
    print("Woof!")

dog = Dog()
dog.speak() # Inherited from Animal
dog.bark()  # Defined in Dog
```

The **Dog** class inherits the **speak** method from **Animal** and adds its own method **bark**. This simplifies code by allowing the **Dog** class to reuse code from **Animal**.

Multiple Inheritance

In multiple inheritance, a class inherits from two or more parent classes. This enables the child class to combine functionality from different sources.

Benefits:

- Provides more flexibility by allowing classes to pull features from multiple sources.
- Useful for creating objects that share behaviour from multiple categories.

Example:

- **Parent Classes:** **Flyable** and **Swimmable**
- **Child Class:** **Duck** inherits from both **Flyable** and **Swimmable**

```
Python
# Parent Classes
class Flyable:
    def fly(self):
        print("I can fly!")

class Swimmable:
```

```
def swim(self):
    print("I can swim!")

# Child Class
class Duck(Flyable, Swimmable):
    pass

duck = Duck()
duck.fly() # Inherited from Flyable
duck.swim() # Inherited from Swimmable
```

The **Duck** class inherits the **fly** method from **Flyable** and the **swim** method from **Swimmable**, enabling it to perform both actions. This is ideal when you want a class to have diverse functionality.

Multilevel Inheritance

Multilevel inheritance creates a chain of inheritance across multiple levels, with each derived class becoming a base class for the next.

Benefits:

- Allows properties and behaviours to be passed down through a sequence of classes.
- Useful for modelling hierarchical relationships where each level adds new functionality.

Example:

- **Base Class:** **Vehicle**
- **Intermediate Class:** **Car** inherits from **Vehicle**
- **Child Class:** **ElectricCar** inherits from **Car**

```

Python

# Base Class
class Vehicle:
    def start(self):
        print("Vehicle starting...")

# Intermediate Class
class Car(Vehicle):
    def drive(self):
        print("Car driving...")

# Child Class
class ElectricCar(Car):
    def charge(self):
        print("Car charging...")

e_car = ElectricCar()
e_car.start() # Inherited from Vehicle
e_car.drive() # Inherited from Car
e_car.charge() # Defined in ElectricCar
  
```

The **ElectricCar** class inherits properties and methods from both **Vehicle** and **Car**, building upon their functionality step-by-step. Each level of inheritance adds specific functionality.

Hierarchical Inheritance

In hierarchical inheritance, multiple child classes inherit from a single parent class. This allows multiple classes to share the same properties and methods from one base class.

Benefits:

- Reduces code duplication by allowing multiple child classes to reuse common functionality.
- Useful when several child classes share the same behaviour but add unique characteristics.

Example:

- **Parent Class:** `Account`
- **Child Classes:** `SavingsAccount` and `CheckingAccount` both inherit from `Account`

```

Python
# Parent Class
class Account:
    def show_balance(self):
        print("Showing balance")

# Child Classes
class SavingsAccount(Account):
    def add_interest(self):
        print("Adding interest")

class CheckingAccount(Account):
    def issue_check(self):
        print("Issuing check")

savings = SavingsAccount()
checking = CheckingAccount()

savings.show_balance() # Inherited from Account
savings.add_interest() # Defined in SavingsAccount

checking.show_balance() # Inherited from Account
checking.issue_check() # Defined in CheckingAccount
  
```

Both `SavingsAccount` and `CheckingAccount` inherit the `show_balance` method from `Account`, but each class also has its own unique methods (`add_interest` and `issue_check`).

Hybrid Inheritance

In **hybrid inheritance**, a combination of multiple inheritance types is used within a class hierarchy, typically involving both single and

multiple inheritance. This allows for more complex relationships where a class may inherit from multiple classes that also have parent-child relationships with each other.

Benefits:

- **Flexible Structure:** Enables classes to inherit attributes and methods from multiple sources, creating versatile class hierarchies.
- **Reusability:** Promotes reuse of common functionality while allowing more specific features in subclasses.
- **Combination of Inheritance Types:** Useful when different inheritance types are needed in one hierarchy.

Example:

- **Parent Class:** `Account`
- **Mixin Class:** `CreditMixin` (provides credit-specific functionality)
- **Child Classes:** `SavingsAccount`, `CheckingAccount`, and `CreditAccount` (inherits from both `Account` and `CreditMixin`).

```
Python
# Parent Class
class Account:
    def show_balance(self):
        print("Showing balance")

# Mixin Class
class CreditMixin:
    def apply_credit_limit(self):
        print("Applying credit limit")

# Child Classes
class SavingsAccount(Account):
    def add_interest(self):
        print("Adding interest")

class CheckingAccount(Account):
    def issue_check(self):
```

```

    print("Issuing check")

class CreditAccount(Account, CreditMixin):
    def pay_credit(self):
        print("Paying credit")

# Testing the classes
savings = SavingsAccount()
checking = CheckingAccount()
credit = CreditAccount()

# Using methods
savings.show_balance()           # Inherited from Account
savings.add_interest()          # Defined in SavingsAccount

checking.show_balance()          # Inherited from Account
checking.issue_check()           # Defined in CheckingAccount

credit.show_balance()            # Inherited from Account
credit.apply_credit_limit()       # Inherited from CreditMixin
credit.pay_credit()              # Defined in CreditAccount

```

- **Account Class:** The parent class with a `show_balance` method, which is shared by all types of accounts.
- **CreditMixin Class:** A mixin class that provides credit-specific functionality (`apply_credit_limit`), not directly related to the primary `Account` functionality.
- **SavingsAccount:** Inherits from `Account` and has its own method `add_interest`.
- **CheckingAccount:** Inherits from `Account` and introduces a unique method `issue_check`.
- **CreditAccount:** Demonstrates hybrid inheritance by inheriting both `Account` and `CreditMixin`, combining general account functionality with credit-specific behaviours.

Hybrid inheritance allows `CreditAccount` to use `show_balance` from `Account` and `apply_credit_limit` from `CreditMixin`, alongside its own `pay_credit` method. This setup is powerful because it allows a class to leverage multiple inheritance sources while maintaining an organised hierarchy.

`super()` in Inheritance

The `super()` function in Python is a powerful tool in inheritance, allowing a child class to access methods and attributes of its parent class. It is commonly used to call the parent class's methods, especially within the `__init__` constructor, to avoid code duplication and ensure that initialization steps from the parent class are inherited in the child class.

Real-World Example: In a workplace model, if "Employee" has properties like name and ID, but a "Manager" requires an additional team size attribute, we can use `super()` to initialise name and ID in the parent class.

```
Python
class Employee:
    def __init__(self, name, id):
        self.name = name
        self.id = id

class Manager(Employee):
    def __init__(self, name, id, team_size):
        super().__init__(name, id) # Calls the parent's __init__
        self.team_size = team_size

manager = Manager("Bob", 202, 8)
print(manager.name)      # Output: Bob
print(manager.team_size) # Output: 8
```

Using `super()` in Different Scenarios

Using `super()` in the `__init__` Constructor

The most common use of `super()` is in a child class's `__init__` method to initialise attributes inherited from the parent class.

Example:

Let's consider an example with a `Vehicle` class and a `Car` class that inherits from it.

```
Python
class Vehicle:
    def __init__(self, fuel_capacity):
        self.fuel_capacity = fuel_capacity
        print("Vehicle initialized with fuel capacity:",
self.fuel_capacity)

class Car(Vehicle):
    def __init__(self, fuel_capacity, model):
        super().__init__(fuel_capacity) # Calls Vehicle's __init__
method
        self.model = model
        print("Car initialized with model:", self.model)

# Creating a Car object
car = Car(50, "Sedan")
```

- The `Car` class calls `super().__init__(fuel_capacity)` to use the parent's initializer.
- `Vehicle's __init__` is executed first, setting `fuel_capacity`, followed by the initialization of `model` in `Car`.

Using `super()` to Call Overridden Methods

If a child class overrides a method from the parent class but still needs to use some functionality from the overridden method, `super()` can be used to call the parent version of that method.

```
Python
class Vehicle:
    def start(self):
        print("Vehicle starting")

class Car(Vehicle):
    def start(self):
        super().start() # Calls Vehicle's start method
        print("Car starting with additional checks")

# Creating a Car object
car = Car()
car.start()
```

- `Car` overrides the `start` method but uses `super().start()` to call the original `start` method from `Vehicle`.
- This allows `Car` to add extra functionality without completely replacing `Vehicle`'s method.

Using `super()` in Multiple Inheritance

When a class inherits from multiple parent classes, `super()` follows the **Method Resolution Order (MRO)**, which determines the order in which parent classes are searched for a method. The MRO is especially useful when dealing with multiple inheritance, as it ensures that methods from each parent are called in a consistent, logical order.

Python

```
class Engine:
    def start(self):
        print("Engine starting")

class Battery:
    def start(self):
        print("Battery powering on")

class HybridCar(Engine, Battery):
    def start(self):
        super().start() # Follows MRO, calling Engine's start method
        print("HybridCar fully started")

# Creating a HybridCar object
car = HybridCar()
car.start()
```

- According to the MRO, `super().start()` first calls `Engine's start` method.
- This setup ensures that `HybridCar` can combine the functionality of multiple parent classes consistently.
- The MRO can be viewed by printing `HybridCar.__mro__`.

Chaining `super()` in Multilevel Inheritance

With multilevel inheritance, `super()` can be used in a chain to call methods up through multiple levels of inheritance.

Python

```
class Animal:
    def sound(self):
        print("Animal sound")

class Mammal(Animal):
    def sound(self):
        super().sound() # Calls Animal's sound method
        print("Mammal sound")
```

```
class Dog(Mammal):
    def sound(self):
        super().sound() # Calls Mammal's sound method
        print("Dog barks")

# Creating a Dog object
dog = Dog()
dog.sound()
```

- **Dog** calls `super().sound()`, which refers to **Mammal's sound**.
- **Mammal** then calls `super().sound()`, referring to **Animal's sound**.
- This **chaining** enables methods to execute across multiple inheritance levels, preserving the behaviour from each ancestor class.

Effective Way for Using `super()`

- **Use `super()` Consistently:** Whenever you override an inherited method and still need the functionality of the parent's version, `super()` should be your first choice.
- **Rely on MRO in Multiple Inheritance:** When using multiple inheritance, let `super()` manage the MRO to avoid hardcoding parent classes and to maintain adaptable code.
- **Understand the MRO Chain:** For complex hierarchies, print the `__mro__` attribute (e.g., `Dog.__mro__`) to understand the sequence in which methods are called.

THANK YOU

